

NAN 2026 GAME x AI 해커톤 · 사전 과제

AI 활용 기술 문서

The Mastermind — 두뇌게임 아레나

플레이 링크 <https://hj0304.github.io/The-Mastermind/>

소스 저장소 <https://github.com/hj0304/The-Mastermind> (공개 · 커밋 기록 유지)

AI 도구 · 프롬프트 · 활용 내역 / 게임 AI 설계 / 외부 에셋·오픈소스 출처

1. 요약 — AI를 두 층위로 활용

본 프로젝트는 AI를 두 층위로 사용했습니다.

- 개발 도구로서의 AI** — Anthropic **Claude**(Claude Code 환경)를 기획·리서치·설계·구현·검증·배포 전 과정의 페어 프로그래머로 사용했습니다. 저장소의 커밋 본문에 `Co-Authored-By: Claude` 로 기여가 표기되어 있습니다.
- 제품 속 AI** — 14종 게임 각각에 맞춤 설계된 대전 AI. 머신러닝 API 호출이 아니라 **게임별 알고리즘**(베이지안 추론, 알파베타 탐색, 행렬게임 혼합전략, 후보 소거 귀납 추론 등)을 순수 TypeScript로 구현했고, 플레이어 성향 학습이 로컬에 누적됩니다. 블라인드 포커는 여기에 더해 **자가대국 강화학습(CFR+)**으로 훈련한 내시 균형 근사 전략을 탑재했습니다.

2. AI 도구 사용 내역 (개발 과정)

항목	내용
도구	Claude Code (Anthropic Claude 모델) — CLI/데스크톱 에이전트 환경
활용 범위	원작 룰 리서치·검증, 게임 룰 명세 작성, 게임 엔진·AI·UI 구현, 온라인 전송 계층 설계, 버그 재현·수정, 브라우저 자동 검증, UI/UX-테마 시스템, 문서 작성
개발 방식	사람이 방향·요구사항·검수를 담당하고, AI가 구현·검증을 수행하는 페어 프로그래밍. GitHub Flow(기능 브랜치 → PR → main 병합)로 모든 변경이 PR 단위로 기록됨
기여 표기	AI가 작성에 참여한 커밋은 본문에 Co-Authored-By로 명시 (커밋 기록에서 확인 가능)

2.1 개발 워크플로우

- 룰 리서치** — 원작(더 지니어스 시리즈 방영 게임)의 룰을 나무위키 원문에서 확인하고, 구현 기준 명세(`docs/GAME_RULES.md`)로 정리. 구현 후에는 원작 문서 13건을 전수 재대조해 불일치를 수정.
- 구현** — 게임마다 엔진(순수 함수)·AI·화면을 분리 구현. 엔진은 부수효과 없는 순수 TypeScript라 AI 탐색과 온라인 동기화에 그대로 재사용.
- 검증** — AI가 브라우저를 직접 조작해 검증: 실제 릴레이를 경유하는 두 탭 온라인 대전, 360px 모바일 뷰 포트 점검, 테마별 스크린샷 확인 등. 빌드는 타입 검사(`tsc`)를 포함.
- 배포** — main 병합 시 GitHub Actions가 GitHub Pages로 자동 배포.

2.2 AI 대상 주요 프롬프트/지시 사항 (실사용 예)

룰 검증 지시

"(나무위키 링크 첨부) 너가 게임 룰을 숙지하고 UI/UX를 수정했는지 의문이 들어. 위 링크를 살살이 뒤져서 모든 게임 룰을 숙지하고 다시 한번 볼래?" — 원작 문서 13건 전수 대조로 이어져, 원작의 '체시 즉시 게이지 갱신' 공개 정보가 온라인 모드에서 누락된 결함을 발견·수정.

버그 리포트

"의심 옷놀이를 플레이했는데 옷판의 정가운데 지점에서 옷이 나왔는데 지름길을 타지 않는 케이스를 발견했어. 지름길을 탈 수 있도록 수정해줘." — 엔진 시뮬레이션으로 로직은 정상임을 확인, 완주 분기가 판 위에 표시되지 않던 UI 문제를 특정해 수정.

설계 요구

"디자인 시안 4종을 각 디자인의 느낌에 맞게 모든 페이지에 적용하고, 토글 버튼으로 유저가 테마를 선택할 수 있게 해줘. 지금 디자인도 저장해서 선택지에 포함해줘." — CSS 변수 기반 5종 테마 시스템으로 구현(게임 의미 색은 불변 원칙).

품질 기준 제시 (프로젝트 지침)

"추측하지 말 것 · 명세와 다르면 멈추고 물을 것 · 요청 범위 밖의 코드를 고치지 말 것 · 작업을 검증 가능한 목표표로 바꿔 통과할 때까지 반복할 것" — 저장소 CLAUDE.md에 상시 지침으로 두어 모든 작업에 적용.

3. 게임 AI 설계 (제품 속 AI)

공정성 원칙 — 모든 AI는 사람과 동일한 공개 정보만 사용합니다. 은닉 정보(상대 손패, 숨은 벽, 주사위 눈)는 코드 수준에서 접근하지 않으며, 각 `ai.ts` 파일 서두에 사용 가능한 정보의 경계를 명문화했습니다.

3.1 게임별 AI 기법

게임	핵심 기법
밀리장기	반복 심화 알파베타 탐색 + 치환표 (시간 예산 900ms, 통상 깊이 9~13)
콰트로	공개 정보 기반 가상 플레이어 손패 추적 + 교환 기대값 평가, 0 카드 주입 방해 전략
블라인드 포커	CFR+ 자가대국 학습(내시 균형 근사) + 카드 카운팅·베이지안 역추론·성향 학습 풀백
윷 대전	행렬게임 혼합전략(동시 선택 심리전) + 사람 선택 분포 학습 + 이동 가치 평가
윷과 거짓말	베이지안 거짓말 확률 추정 + 사람의 거짓 빈도·의심률 학습
리플렉트	반복 심화 알파베타 + 치환표, 레이저-왕 근접도 평가로 전진 동기 부여
모노크롬	베이지안 손패 추적 + 상대 신념 모델링(상대가 내 정보로 무엇을 아는지) + 종반 완전 탐색 + 성향 학습
모노크롬 II	잔여 포인트 구간 추적 + 점수 압박 기반 입찰(블로토 게임 이론)
야누스 포커	카드 카운팅 + 홀짝 제약 추론 + 베이지안 뒷면 추정 + 기대값 베틱
암전 미궁	공개된 충돌/통과 정보 완전 기억 + 미지 간선의 확률 기대비용 최단경로 탐색
순환선	순환 완성 가능성 판정(증인 사이클) + 메모이제이션 승패 탐색(벽시계 예산 2.5초)
모노크롬 레이즈	블로토식 배치 설계 + 칩 신호 기반 타일 추론 + 콜/폴드 기대값
수(數)의 진	은폐 기물 정체 확률 분포 + "움직인 기물은 지뢰가 아니다" 이동 이력 추론 + 위협 분석
히든 포물러	규칙 은행 후보를 힌트와 대조해 소거하는 베이지안 귀납 추론 — 확정 시에만 버저

3.2 자가대국 강화학습 — 블라인드 포커 CFR+

포커 AI(Libratus/Pluribus)가 인간을 이긴 기법인 **CFR(Counterfactual Regret Minimization)** 계열을 사용해, 블라인드 포커의 베틱 전략을 자가대국으로 학습했습니다.

- **모델링** — 한 핸드를 불완전정보 게임으로 정확히 정식화: 20장 덱의 모든 딜을 확률 가중 열거하고, 정보집합 = (내가 보는 상대 카드, 베틱 히스토리). 레이즈 추상화는 실제 게임의 행동 공간($\{+1, +3, +5, \text{올인}\}$)과 동일해 손실이 없습니다. 10 폴드 페널티도 기대값에 정확히 반영됩니다.
- **학습** — CFR+(regret matching+, 선형 가중 평균)로 6,000회 반복, **정보집합 14,560개**의 평균 전략이 내시 균형에 수렴. GPU 없이 로컬에서 약 8분 소요. 학습 스크립트(`scripts/cfr/`)와 재현 커맨드(`npm run cfr:train`)를 저장소에 포함했습니다.

- **평가** — 학습 전략 vs 기존 휴리스틱 AI(카운팅+베이지안)를 실제 엔진에서 4,000판 대결시킨 결과 **승률 70.2%** (정보집합 적중률 100%). 게임에는 학습 전략표(JSON)를 지연 로드해 탑재하고, 미로드 시 휴리스틱으로 폴백합니다.
- **의미** — 균형 전략은 원리상 착취당하지 않는(unexploitable) 플레이를 합니다. 확률적 블러핑·10 페널티 회피 베팅 같은 고급 패턴이 학습 과정에서 저절로 창발했습니다.

3.3 대표 사례 심화

모노크롬 — **상대 신념 모델링**. 상대의 남은 타일 분포를 색·승패 기록으로 베이지안 갱신하는 데 그치지 않고, "상대가 내 공개 정보로 무엇을 알고 있는가"까지 모델링해 블러핑 여지를 계산합니다. 라운드별 제시 랭크 분포는 localStorage에 누적되어 판을 거듭할수록 개인화됩니다.

히든 포물러 — **오답 없는 귀납 추론**. AI는 숨은 규칙을 읽지 않습니다. 규칙 은행의 후보들을 힌트와 대조해 소거하고, 후보가 하나로 확정되거나 모든 후보의 답이 일치할 때만 버저를 누릅니다. 수 제시는 후보 판별력을 최대화하는 능동 실험으로 선택합니다 — 사람과 같은 방식으로 추론하되, 실수만 없는 상대입니다.

윷과 거짓말 — **성향 학습**. 의심으로 공개된 선언만을 근거로 사람의 거짓말 빈도와 값별 분포, 값별 의심률을 학습해 베イズ 추정에 반영합니다. 믿어준 선언의 실제 값은 AI도 영원히 알 수 없습니다(공정성 원칙).

4. 시스템 아키텍처

- **스택** — TypeScript + React 19 + Vite. 게임 엔진·AI는 외부 라이브러리 없이 순수 TypeScript.
- **구조** — 게임마다 `engine.ts` (순수 함수 룰) / `ai.ts` (대전 AI) / `view.ts` (좌석별 정보 마스킹) / 화면 컴포넌트로 분리.
- **온라인 멀티** — 서버 없이 공개 nostr 릴레이를 데이터 통로로 사용(방 코드에서 유도한 키로 AES-GCM 암호화). 호스트 권위 방식 + 좌석별 관점 뷰만 전송. 재연결·메시지 순번 순서 보장·재전송 유실 복구·좌석 고정을 전송 계층에서 처리.
- **치팅 방지** — 동시 선택·비밀 설계는 **커밋-리빌**(SHA-256 해시 커밋 후 공개·검증)로, 주사위는 **분산 생성**(굴리는 쪽 난수 커밋 + 상대 난수 합산)으로 호스트조차 엿볼 수 없게 설계.
- **테마 시스템** — CSS 변수 토큰 기반 5종 테마. 게임 의미 색(흑/백 타일, 4색 카드, 레이저 등)은 테마와 무관하게 불변으로 두어 룰 가독성을 보존.

5. 외부 에셋 / 오픈소스 출처

구분	이름	라이선스 / 비고
라이브러리	React 19, React DOM	MIT
라이브러리	Vite, @vitejs/plugin-react, TypeScript	MIT / Apache-2.0 (빌드 도구)
라이브러리	nostr-tools	MIT — 온라인 대전 이벤트 서명/검증
인프라	공개 nostr 릴레이 (relay.damus.io, nos.lol, relay.primal.net)	공개 릴레이 — 게임 데이터는 암호화되어 경유만 함
폰트	Pretendard Variable	SIL OFL 1.1

폰트	Space Grotesk · Chakra Petch · Orbitron · Cinzel (Google Fonts)	SIL OFL 1.1 — 테마별 디스플레이 폰트
그래픽	이모지(게임 아이콘 등)	플랫폼 기본 이모지 — 별도 이미지 에셋 없음
그래픽·사운드	기타 이미지/사운드 에셋	사용하지 않음 — 모든 비주얼은 CSS/SVG로 직접 제작

룰 출처 표기

각 게임은 더 지니어스 시리즈 방영 게임의 룰(나무위키 '더 지니어스/역대 게임' 및 개별 게임 문서에서 확인)과 그 원류 보드게임(동물장기, Khet, 베니스 커넥션, 마법의 미로, 찰오찰오, 스트라테고, 파라오코드 등)을 참고해 **룰을 재해석·재구현**한 것입니다. 원작의 그래픽·명칭·소스는 일절 사용하지 않았으며, 게임 이름도 모두 새로 지었습니다. 변형한 규칙은 게임 내 설명서와 `docs/GAME_RULES.md`에 원작과의 차이로 명시되어 있습니다.